PROGRAMMING LANGUAGE

v1.0.0

EHX Lang

The Complete **Programming Book**

REFERENCE · GUIDE · CHEAT SHEET

MAIN.EHX

```
set secure = true

when ready:
    show("Build")

fn create_future():
```

CODE • BUILD • SECURE • GROW

CREATED & DESIGNED BY

Yonatan Muluken

ETHIO HACKER X · @EthioHackerX

EHX Lang

The Complete Programming Book

Reference, Guide & Cheat Sheet

VERSION 1.0.0

Yonatan Muluken

ETHIO HACKER X

CONTENTS

A guided route through the language

How to Use This Book 4

PART I · Getting Started 6

01. Meet EHXLang 7

02. Installation & Setup 9

03. Mission, Vision & Design Goals 11

PART II · The Core Language 12

04. Variables, F-Strings & Math 13

05. Decisions & Loops 15

06. Functions & Modules 17

07. Console Input, Output & Formatting 19

08. Types, Inspection & Conversion 21

09. Numbers & Mathematics 23

PART III · Working with Data 25

10. Lists & Arrays 26

11. Strings 29

12. Dictionaries 31

13. Cryptography, Encoding & JSON 32

PART IV · Systems & Networking 34

14. Files, System State & Time 35

15. Archives & Regular Expressions 38
16. Networking & HTTP 40
17. Concurrency & Flow Helpers 42

PART V · Practice 44
18. Practical Patterns & Safety 45
19. Mini Projects 47

A. Complete Function Reference 50

B. Quick Lookup Index 57

C. Glossary 58

BEFORE YOU BEGIN

How to Use This Book

A short map of the book, and the visual language it uses to guide you.

This book is both a **learning companion** and a **desk reference**. You can read it from front to back, or jump straight to the chapter that answers your question. Every topic follows the same rhythm: the purpose of the feature, its syntax, a small runnable example, and a practical note.

How the book is organized

PART	WHAT YOU WILL LEARN
I · Getting Started	What EHXLang is, how to install it, and what it is designed to do.
II · The Core Language	Variables, f-strings, decisions, loops, functions, modules, console I/O, types and math.
III · Working with Data	Lists, strings, dictionaries, hashing, encoding and JSON.
IV · Systems & Networking	Files, the operating system, archives, regular expressions, HTTP, sockets and concurrency.
V · Practice	Patterns, safety, and four complete mini projects.
Appendices	The complete function reference, a quick-lookup index and a glossary.

Reading the page

Code appears in dark windows with colour-coded syntax. Results appear in light windows labelled **OUTPUT**.

SYNTAX.EHX

```
# Keywords, built-in functions, strings, numbers and comments
set port = 8080
when port == 8080:
    show("Alternate HTTP port")
```

TIP

Tips point out shortcuts and good habits.

NOTE

Notes add background or explain how something behaves.

SAFETY

Safety boxes mark features that touch real systems, files or networks.

TRY IT

Try It boxes give you a small challenge to type in and run yourself.

A note on the examples

Every function name, method name and signature in this book comes from the EHXLang v1.0.0 reference. Examples are intentionally small so that errors are easy to locate. Type them yourself rather than pasting them; you will learn the syntax faster.

PART I

Getting Started

Meet the language, install it, and understand what it was built to do.

IN THIS PART

01 Meet EHXLang

02 Installation & Setup

03 Mission, Vision & Design Goals

CHAPTER 01

Meet EHXLang

A modern, human-readable, general-purpose programming language for everyone.

EHXLang replaces complex syntax and academic jargon with clear, active, plain-English instructions. You create a variable with `set`, branch with `when`, show output with `show`, and write a file with `save`. The result is code that reads almost like a sentence.

Behind the scenes, the interpreter engine is written in Python. It parses your source into an Abstract Syntax Tree (AST), analyzes it, and executes it in a runtime environment, so the same program runs on Windows, Linux and macOS.

Your first program

EXAMPLE.EHX

```
set user = "Yonatan"
set port = 8080

show(f"Hello, {user}! Listening on port {port}.")
```

Read it aloud: *set* `user` to a name, *set* `port` to a number, then *show* a message. The `f` before the quotes turns the string into an **f-string**, so `{user}` and `{port}` are replaced with their values.

What you can build

The v1.0.0 toolkit combines everyday scripting with network and security-oriented primitives in a single language. Together they cover:

- **Scripting and automation:** files, environment variables, shell commands, timers.
- **Data handling:** lists, strings, dictionaries, JSON, regular expressions, archives.
- **Networking:** HTTP requests and raw TCP sockets.
- **Security-related tooling:** MD5, SHA-1, SHA-256, SHA-512 and Base64.
- **Concurrency:** background tasks, timeouts and intervals.

The language at a glance

The table below is the signature of EHXLang. Each task has one direct, memorable spelling.

TASK	EHXLANG	WHY IT FEELS NATURAL
Declare a variable	`set x = 10`	An active instruction keyword
Write to a file	`save("file.txt", "Data")`	One call, no manual file handles
Read from a file	`set c = read("file.txt")`	Direct, with no file-leak risk
Append to a file	`add_to("file.txt", "Log\n")`	Plain English
Check a file exists	`when has_file("conf.json"):`	A direct condition
Read file lines	`set l = lines("words.txt")`	Loads lines straight into a list
Display output	`show(...)`	Punchy standard output
Ask the user	`set u = ask("Name: ")`	Interactive and readable
Branching	`when / or when / otherwise`	Reads like spoken English
Import a module	`load "math_lib.ehx"`	Distinct, clean loading
Pause	`pause(2)`	A direct delay
Background thread	`set t = spawn(my_worker)`	Native concurrency
Inspect a type	`kind(x)`	Returns "string", "int", ...
List append / remove	`ports.add(8080) / ports.rm()`	Single-purpose methods
List sort / join	`ports.order() / ports.fuse(" | ")`	Signature method names
String interpolation	`f"Host: {host}:{port}"`	Variables and math inside text

THE CREATOR

Created by Yonatan Muluken (ETHIO HACKER X), creator and lead architect of EHXLang. Telegram and YouTube `@EthioHackerX` · TikTok `@ethio_hacker_x` · GitHub `github.com/EthioHackerX`

CHAPTER 02

Installation & Setup

Install the runtime, run your first script and open the interactive shell.

You can install EHXLang from the pre-built binaries on the GitHub Releases page, or directly from the source repository. Either way, you end up with a single command: `ehx`.

Option A: Windows

Download `EHXSetup.exe` from the latest release and run it. The installer works in four stages:

STAGE	WHAT HAPPENS
1 · Privilege mode	Automatically detects administrator or current-user permissions.
2 · Setup wizard	Review the license, choose an installation folder, and select components.
3 · Configuration	Adds `ehx` to your system PATH and installs the official VS Code extension.
4 · Instant access	Open any Command Prompt or PowerShell window and type `ehx`.

Option B: Linux & macOS

One short sequence installs everything:

TERMINAL

```
$ git clone https://github.com/EthioHackerX/EHXLang.git
$ cd EHXLang
$ chmod +x install.sh
$ ./install.sh
```

The script installs the runtime to `~/.local/share/ehxlang`, links the `ehx` launcher into `~/.local/bin/ehx`, deploys the VS Code extension to `~/.vscode/extensions/`, and adds `~/.local/bin` to your PATH in `.bashrc` or `.zshrc`.

The ehx command

COMMAND	PURPOSE
`ehx main.ehx`	Run a script.
`ehx run main.ehx`	Run a script (long form).
`ehx repl`	Start the interactive REPL shell.
`ehx -v`	Display the version and banner.

TERMINAL

```
$ ehx main.ehx
$ ehx repl
$ ehx -v
```

VS Code extension

The official Visual Studio Code extension gives you **syntax highlighting** for keywords (`set`, `when`, `fn`, `load`), built-in functions and f-strings, plus **smart indentation** that applies four spaces after you type a colon. The Windows installer deploys it automatically to `~/.vscode/extensions/ehx-vscode`.

TIP

Blocks open with a colon and are indented four spaces. Save your files with the `.ehx` extension.

TRY IT

Create `hello.ehx` containing `show("Hello, EHXLang!")` and run it with `ehx hello.ehx`. Then start `ehx repl` and type the same line.

CHAPTER

03 Mission, Vision & Design Goals

Why EHXLang exists, and the principles that shape it.

Mission

To make practical programming more approachable by bringing useful scripting, automation, data handling, and network-related tools into one clear learning experience, while encouraging responsible, secure, and understandable code.

Vision

To grow EHXLang into a recognizable, community-supported programming project that helps learners and builders turn ideas into useful tools, learn by experimenting, and contribute to practical software projects.

Design goals

Readable examples

Show a clear task and a small piece of code that demonstrates it.

Useful built-ins

Keep printing, conversion, file access, hashing and JSON easy to find.

Practical scope

Support learning across scripting, automation, networking and security.

Responsible use

Use network and system features only where you have permission.

What makes EHXLang different?

Its distinguishing emphasis is general scripting combined with network and security-oriented primitives: HTTP helpers, TCP sockets, hashing, file operations and background tasks, all in one plain-English style.

PART II

The Core Language

Variables, decisions, loops, functions and the everyday toolbox every program needs.

IN THIS PART

04 Variables, F-Strings & Math
05 Decisions & Loops
06 Functions & Modules
07 Console Input, Output & Formatting
08 Types, Inspection & Conversion
09 Numbers & Mathematics

CHAPTER 04

Variables, F-Strings & Math

Store values, build messages from them, and calculate.

Declaring variables

Use `set` to create a variable. The name goes on the left, the value on the right.

EXAMPLE.EHX

```
set user = "Yonatan"
set target = "192.168.1.50"
set port = 8080
```

F-strings

Put an `f` in front of a string and write variables or whole expressions inside curly braces. EHXLang evaluates them and inserts the result.

EXAMPLE.EHX

```
# Clean string interpolation
show(f"Connecting to {target}:{port} for {user}...")
```

Mathematics

Math works inside and outside f-strings. Three operators deserve special attention:

OPERATOR	MEANING	EXAMPLE	RESULT
`**`	Exponentiation	`2 ** 8`	256
`//`	Integer floor division	`17 // 5`	3
`%`	Modulo (remainder)	`17 % 5`	2

EXAMPLE.EHX

```
# Math operations: exponentiation, floor division, modulo
show(f"2 ** 8 = {2 ** 8}")       # 256
show(f"17 // 5 = {17 // 5}")     # 3
show(f"17 % 5  = {17 % 5}")      # 2

# Finding extremes
show(f"Lowest: {low(10, 5, 20)} | Highest: {high(10, 5, 20)}")
```

NOTE

`low` and `high` are shorthand aliases for `lowest` and `highest`. You will meet them again in Chapter 9.

TRY IT

Set a variable `bits = 16` and use an f-string to print how many values a key of that many bits can hold, using `2 ** bits`.

KEY TAKEAWAYS

- `set name = value` creates a variable.
- `f"...{expression}..."` embeds values and math inside text.
- `**`, `//` and `%` give you power, floor division and remainder.

CHAPTER

Decisions & Loops

05

Make your program choose, repeat and recover from errors.

when, or when, otherwise

Branching reads like natural speech. Start with `when`, add as many `or when` branches as you like, and finish with an optional `otherwise`. Every branch ends with a colon and an indented block.

EXAMPLE.EHX

```
set status_code = 200

when status_code == 200:
    show("Request Successful!")
or when status_code == 404:
    show("Endpoint Not Found.")
otherwise:
    show("Unknown status code:", status_code)
```

Use `not` to reverse a condition:

EXAMPLE.EHX

```
set connection_status = false

when not connection_status:
    show_err("CRITICAL: Failed to bind port 443. Access denied.")
```

for loops

A `for` loop visits every item in a list:

EXAMPLE.EHX

```
set active_ports = [21, 22, 80, 443]

for p in active_ports:
    show("Auditing port:", p)
```

Looping over numbers with range

`range(start, stop)` generates a list of numbers. The stop value itself is not included, so `range(1, 4)` produces 1, 2 and 3. You can also give an optional step.

EXAMPLE.EHX

```
for i in range(1, 4):
    show("Retry attempt:", i)
```

while loops

A `while` loop repeats as long as its condition holds. This one waits for a background task (see Chapter 17):

EXAMPLE.EHX

```
while not scan_task.is_done():
    show("Working in background...")
    pause(0.2)
```

try and catch

Operations such as opening a network connection can fail. Wrap them in `try`, and handle the problem in `catch`, which receives the error.

EXAMPLE.EHX

```
try:
    set sock = tcp_connect("127.0.0.1", 80, 3.0)
    sock.close()
catch err:
    show_err("Connection failed:", err)
```

TRY IT

Write a loop with `range(1, 6)` that prints each number, but only when it is greater than 2 using a `when` block inside the loop.

KEY TAKEAWAYS

- `when` / `or when` / `otherwise` handle every decision.
- `for` iterates lists and ranges; `while` repeats until a condition changes.
- `try` / `catch` keeps failures from stopping your program.

CHAPTER

06 Functions & Modules

Package logic once, reuse it everywhere, and split code across files.

Defining functions

Declare a function with `fn`, give it parameters, and send a value back with `return`.

EXAMPLE.EHX

```
fn double_val(x):
    return x * 2

show(double_val(21))
```

Functions are values

You can hand a function to another function without calling it. This is how `map`, `filter`, `for_each`, `spawn`, `set_timeout` and `set_interval` work: you pass the name only, with no parentheses.

EXAMPLE.EHX

```
set ports = [8080, 22, 443]
set doubled_ports = ports.map(double_val)
show("Doubled ports:", doubled_ports)
```

Modules

When a program grows, move reusable functions into their own file and bring them in with `load`.

MATH_UTILS.EHX

```
# In math_utils.ehx:
fn calculate_hash_space(bits):
    return 2 ** bits
```

MAIN.EHX

```
# In main.ehx:
load "math_utils.ehx"

show(f"Keyspace for a 16-bit key: {calculate_hash_space(16)}")
```

TIP

Give functions verbs for names (`calculate_hash_space`, `check_port`). A function that does one thing is easy to test, rename and reuse.

TRY IT

Create `utils.ehx` with a function `greet(name)` that returns a message. Load it from `main.ehx` and show the result.

KEY TAKEAWAYS

- `fn name(params):` defines a function; `return` hands back a value.
- Functions can be passed by name into `map`, `filter`, `spawn` and timers.
- `load "file.ehx"` brings another file's functions into your program.

CHAPTER

07

Console Input, Output & Formatting

Talk to the person running your script.

Console functions display information, accept interactive input, read command-line arguments and build formatted messages.

`show(...args)`

Prints values to standard output, separated by spaces, followed by a newline.

EXAMPLE.EHX

```
set host = "192.168.1.10"
set port = 80
show("Target established at:", host, "on port", port)
```

`show_err(...args)`

Writes diagnostic or error messages to standard error.

EXAMPLE.EHX

```
set connection_status = false
when not connection_status:
    show_err("CRITICAL: Failed to bind port 443. Access denied.")
```

`ask(prompt="")`

Reads keyboard input after displaying an optional prompt.

EXAMPLE.EHX

```
set username = ask("Enter administrator username: ")
show("Welcome back,", username)
```

`args()`

Returns a list of the command-line arguments passed to the script.

EXAMPLE.EHX

```
set cli_flags = args()
show("Script argument count:", len(cli_flags))
```

`format(template, ...args)`

Replaces sequential `{}` placeholders in a template with the supplied arguments.

EXAMPLE.EHX

```
set message = format("Connecting to IP: {}, Port: {}", "10.0.0.5", 22)
show(message)
```

OUTPUT

```
Connecting to IP: 10.0.0.5, Port: 22
```

NOTE

`format` and f-strings solve the same problem. Use `format` when the template comes from somewhere else, such as a file or a variable; use f-strings when the text is written inline.

TRY IT

Ask for the user's name with `ask`, then greet them using `format`.

CHAPTER 08

Types, Inspection & Conversion

Know what a value is, and turn it into what you need.

Every value has a type. Inspection helps a script understand what it received; conversion turns it into the type an operation needs.

Type names

TYPE	EXAMPLE
`string`	`"hello"`
`int`	`8080`
`float`	`3.14`
`bool`	`true / false`
`list`	`[80, 443, 8080]`
`dict`	`{"host": "127.0.0.1"}`
`none`	`the absence of a value`

`kind(val)`

Returns the type name of a value as a string, such as "string", "int" or "list".

EXAMPLE.EHX

```
set payload = [80, 443, 8080]
when kind(payload) == "list":
    show("List length:", len(payload))
```

len(val)

Returns the number of characters in a string, items in a list, or pairs in a dictionary.

EXAMPLE.EHX

```
set targets = ["127.0.0.1", "192.168.1.1"]
set token = "secret_auth_token_xyz"
show("Targets:", len(targets))
show("Token characters:", len(token))
```

str(val) int(val) float(val) bool(val)

Explicit conversion functions for scalar values.

EXAMPLE.EHX

```
set port_str = "8080"
set port_num = int(port_str)      # 8080
set ratio = float("3.14159")      # 3.14159
set flag = bool(1)                # true
set text_repr = str(port_num)     # "8080"
show(port_num, ratio, flag, text_repr)
```

is_nan(val)

Checks whether a value is not a valid number. Use it to validate input before converting.

EXAMPLE.EHX

```
set input_val = "abc123_invalid"
when is_nan(input_val):
    show_err("Validation Error: Input is not a valid number format.")
```

TIP

`ask` always gives you text. Convert it with `int` or `float` before doing math, and check with `is_nan` first when the input is not trusted.

TRY IT

Use `kind` to print the type of `42`, `"42"`, `4.2` and `[4, 2]`.

CHAPTER 09

Numbers & Mathematics

Find extremes, round, take roots and generate random numbers.

Extremes

`lowest(...args | list) low(...)`

Finds the smallest value. Pass several arguments or a single list. `low` is a shorthand alias.

EXAMPLE.EHX

```
set latencies = [45, 12, 89, 23, 7]
show("Lowest latency:", lowest(latencies))   # 7
show(low(50, 10, 80))                        # 10
```

`highest(...args | list) high(...)`

Finds the largest value, using the same argument styles. `high` is the alias.

EXAMPLE.EHX

```
show("Highest latency:", highest(45, 12, 89, 23, 7))   # 89
show(high([50, 10, 80]))                               # 80
```

Math helpers

FUNCTION	DESCRIPTION	EXAMPLE
`math_abs(x)`	Absolute value	`math_abs(-42) # 42`
`math_sqrt(x)`	Square root of a non-negative number	`math_sqrt(64) # 8.0`
`math_pow(base, exp)`	Raises base to an exponent	`math_pow(2, 8) # 256.0`
`math_floor(x)`	Rounds down to the nearest integer	`math_floor(4.9) # 4`
`math_ceil(x)`	Rounds up to the nearest integer	`math_ceil(4.1) # 5`
`math_round(x)`	Rounds to the nearest whole integer	`math_round(4.6) # 5`

EXAMPLE.EHX

```
set absolute_diff = math_abs(-150)
set root = math_sqrt(144)
set power_val = math_pow(2, 10)
set rounded = math_round(4.678)
set floored = math_floor(4.99)
set ceiled = math_ceil(4.01)
show(power_val, rounded)
```

Random numbers

FUNCTION	DESCRIPTION	EXAMPLE
`math_random(low=0, high=1)`	Random decimal in the range [low, high]	`math_random(0, 10)`
`math_randint(low, high)`	Random integer, inclusive of both ends	`math_randint(1000, 9999)`

SAFETY

Random helpers are fine for games, samples and test data. Do not rely on them alone to generate secrets such as passwords or keys.

TRY IT

Roll a die with `math_randint(1, 6)` ten times using a `for` loop and `range`, and print the highest roll with `highest`.

PART III

Working with Data

Lists, strings, dictionaries, hashes, encodings and JSON.

IN THIS PART

10 Lists & Arrays
11 Strings
12 Dictionaries
13 Cryptography, Encoding & JSON

CHAPTER 10

Lists & Arrays

Collections you can grow, shrink, sort, search and transform.

A list is written with square brackets, such as `[80, 21, 443]`. List methods modify, search, order or transform the collection.

`.add(item)`

Appends an item to the end of the list.

EXAMPLE.EHX

```
set ports = [80, 21, 443]
ports.add(8080)
```

`.rm()`

Removes the last element and returns it.

EXAMPLE.EHX

```
set ports = [80, 21, 443, 8080]
set popped = ports.rm()
show("Removed:", popped)     # 8080
```

`.order()`

Sorts the list in place.

EXAMPLE.EHX

```
set ports = [80, 21, 443]
ports.order()
show(ports)
```

`.fuse(sep="")`

Joins the items into a single string using a separator.

EXAMPLE.EHX

```
set ports = [21, 80, 443]
show(ports.fuse(" | "))      # 21 | 80 | 443
```

`.contains(item) .reverse()`

Check membership, or reverse the list in place.

EXAMPLE.EHX

```
set nums = [1, 2, 3, 4, 5]
when nums.contains(3):
    show("Collection contains 3")
nums.reverse()
```

`.slice(start, [end], [step])`

Extracts a sub-list.

EXAMPLE.EHX

```
set ports = [21, 22, 80, 443, 8080]
show(ports.slice(0, 3))
```

Functional helpers

`map`, `filter` and `for_each` take a function. `map` transforms every element, `filter` keeps elements for which the function returns `true`, and `for_each` simply runs the function for each element.

EXAMPLE.EHX

```
fn double(x):
    return x * 2

fn is_even(x):
    return x % 2 == 0

set nums = [1, 2, 3, 4, 5]
set doubled_nums = nums.map(double)
set even_nums = nums.filter(is_even)

show("Doubled:", doubled_nums)
show("Evens:", even_nums)
nums.for_each(show)
```

Quick reference

METHOD	WHAT IT DOES
`.add(item)`	Append an item
`.rm()`	Remove and return the last element
`.order()`	Sort in place
`.fuse(sep)`	Join into a string
`.len()`	Count the elements
`.clear()`	Remove everything
`.contains(item)`	true if the item is present
`.reverse()`	Reverse in place
`.slice(s, e, step)`	Extract a sub-list
`.map(fn)`	Transform every element
`.filter(fn)`	Keep elements where fn returns true
`.for_each(fn)`	Run fn for every element

TRY IT

Build a list of five numbers, keep only those above 10 with `filter`, then join the result with `fuse(", ")` and show it.

CHAPTER 11

Strings

Search, slice, clean and reshape text.

Strings are written between quotes. Their methods search, count, change case, trim whitespace, split text and replace substrings.

`.find(sub)` `.count(sub)`

`find` returns the zero-based index of a substring, or `-1` when it is missing. `count` returns how many times it occurs.

EXAMPLE.EHX

```
show("ehx lang".find("lang"))    # 4
show("banana".count("a"))        # 3
```

`.upper()` `.lower()` `.trim()`

Change letter case, or strip leading and trailing whitespace.

EXAMPLE.EHX

```
set raw_text = "  EHXLang Programming Language  "
set clean_text = raw_text.trim().upper()
show(clean_text)
```

`.split(sep=" ")`

Splits a string into a list at every separator.

EXAMPLE.EHX

```
set ip_str = "192.168.1.1"
set octets = ip_str.split(".")
show(octets)
```

`.replace(old, new)`

Replaces every occurrence of a substring.

EXAMPLE.EHX

```
show(ip_str.replace(".", "-"))   # 192-168-1-1
```

`.contains(sub) .starts_with(sub) .ends_with(sub)`

Boolean checks on the contents of a string.

EXAMPLE.EHX

```
when url.starts_with("https"):
    show("Secure URL")
when "admin:1234".contains("admin"):
    show("Contains admin")
```

`.slice(start, [end], [step])`

Extracts a substring.

EXAMPLE.EHX

```
show("hello".slice(0, 4))        # hell
```

TIP

Methods can be chained: `raw.trim().upper()` runs left to right, so each step receives the result of the one before it.

TRY IT

Given `set line = " user=admin;role=owner "`, trim it, split it on `;`, and show each part.

CHAPTER

Dictionaries

12

Named values: look up data by key instead of position.

A dictionary stores key and value pairs inside curly braces. Read a value by writing its key in square brackets after the dictionary name.

EXAMPLE.EHX

```
set config = {"host": "127.0.0.1", "port": 443}

show("Config Keys:", config.keys())
when config.has("port"):
    show("Port config is present:", config["port"])
```

METHOD	WHAT IT DOES	EXAMPLE
`.keys()`	List of all keys	`config.keys()`
`.values()`	List of all values	`config.values()`
`.len()`	Number of key-value pairs	`config.len()`
`.has(key)`	true if the key exists	`config.has("port")`
`.clear()`	Remove all keys and values	`config.clear()`

TIP

Check `has(key)` before reading a key that might be missing, especially for data that comes from JSON or user input.

TRY IT

Create a dictionary describing a server (`name`, `ip`, `port`), then print its keys and its length.

CHAPTER 13

Cryptography, Encoding & JSON

Hash data, encode text, and move structured values in and out of text.

> **NOTE**
>
> **Encoding is not hashing.** Base64 is a reversible *encoding*. MD5 and the SHA family produce one-way *hashes*. Neither is a substitute for proper password storage.

Hashing

Four hashing functions are built in. Each takes text and returns a hexadecimal digest.

FUNCTION	ALGORITHM
`md5(text)`	MD5
`sha1(text)`	SHA-1
`sha256(text)`	SHA-256
`sha512(text)`	SHA-512

EXAMPLE.EHX

```
set sample_text = "EthioHackerX_Secure2026"
show("MD5:    ", md5(sample_text))
show("SHA-1:  ", sha1(sample_text))
show("SHA-256:", sha256(sample_text))
show("SHA-512:", sha512(sample_text))
```

Base64

EXAMPLE.EHX

```
set token = "SecretKey123"
set encoded = base64_encode(token)
set decoded = base64_decode(encoded)
show("Encoded:", encoded)
show("Decoded:", decoded)
```

JSON

`to_json(data, indent)` turns a list or dictionary into JSON text; the optional indent makes it readable. `from_json(text)` parses JSON text back into a dictionary or list.

EXAMPLE.EHX

```
set payload = {"status": 200, "access": true}
set json_str = to_json(payload, 2)
set parsed_dict = from_json(json_str)
show("JSON payload serialized successfully.")
show(parsed_dict["status"])
```

SAFETY

MD5 and SHA-1 are fine for checksums and learning exercises, but they are not appropriate for protecting passwords. Never place real secrets in example code.

TRY IT

Hash the word `ehx` with `sha256`, then change one letter and hash it again. Compare the two results.

KEY TAKEAWAYS

- Hash functions are one-way; Base64 is reversible.
- `to_json` and `from_json` convert between values and JSON text.
- Hashes help verify that data has not changed.

PART IV

Systems & Networking

Files, the operating system, archives, patterns, the network and concurrency.

IN THIS PART

14 Files, System State & Time

15 Archives & Regular Expressions

16 Networking & HTTP

17 Concurrency & Flow Helpers

CHAPTER

14 Files, System State & Time

Read and write files, run commands, and measure time.

File and system helpers are the backbone of small automation scripts, logs, configuration and timing.

`save(path, data)`

Writes data to a file, creating it or overwriting what was there.

EXAMPLE.EHX

```
save("audit.log", "[INIT] System booted successfully\n")
```

`add_to(path, data)`

Appends data to the end of an existing file.

EXAMPLE.EHX

```
add_to("audit.log", "[EVENT] User authentication verified\n")
```

`read(path)`

Reads the complete contents of a file as text.

EXAMPLE.EHX

```
set log_contents = read("audit.log")
show(log_contents)
```

`has_file(path) lines(path)`

`has_file` returns `true` if the file exists. `lines` reads a file into a list with one string per line.

EXAMPLE.EHX

```
when has_file("config.json"):
    set word_list = lines("wordlist.txt")
    show("Loaded lines:", len(word_list))
```

The system around your script

`pause(seconds)`

Pauses execution for the given number of seconds. Decimals are allowed.

EXAMPLE.EHX

```
pause(1.5)
```

`exec(command)`

Runs an operating-system command and returns a dictionary with `code`, `stdout` and `stderr`.

EXAMPLE.EHX

```
set result = exec("whoami")
show("Current User Output:", result["stdout"])
```

`env(key, default="")`

Reads an environment variable, falling back to a default when it is not set.

EXAMPLE.EHX

```
set current_user = env("USER", "root_admin")
```

time_now() timestamp()

`time_now` returns the current date and time as formatted text. `timestamp` returns the current epoch time as a float, which is ideal for measuring how long something takes.

EXAMPLE.EHX

```
set start = timestamp()
show("Time:", time_now())
pause(1)
show("Elapsed seconds:", timestamp() - start)
```

SAFETY

`exec` runs real commands on your machine. Review every command before running it, and never build a command from untrusted input.

TRY IT

Write a script that saves a one-line note to `notes.txt`, appends a second line with `add_to`, and prints the file back with `read`.

CHAPTER 15

Archives & Regular Expressions

Package files into ZIPs and match text against patterns.

ZIP archives

```
zip_pack(source, zip_path)
```

Compresses a file or an entire directory into a `.zip` archive.

EXAMPLE.EHX

```
zip_pack("logs_dir/", "system_backup.zip")
```

```
zip_unpack(zip_path, dir)
```

Extracts a `.zip` archive into a target folder.

EXAMPLE.EHX

```
zip_unpack("system_backup.zip", "restored_logs/")
```

Regular expressions

A regular expression describes a pattern of text. EHXLang provides three helpers:

FUNCTION	RETURNS
`regex_match(pattern, text)`	`true` if the pattern matches the text
`regex_find(pattern, text)`	A list of every matching substring
`regex_replace(pat, rep, txt)`	The text with every match replaced

EXAMPLE.EHX

```
set ip_address = "192.168.1.50"

when regex_match("^\\d{1,3}\\.\\d{1,3}", ip_address):
    show("IP address format matched the pattern.")

set versions = regex_find("\\d+\\.\\d+", "ehx 1.0 and lib 2.4")
set masked = regex_replace("[0-9]", "#", "pin 4821")
```

NOTE

Inside a normal string, a backslash must be doubled so that the regex engine receives a single backslash. That is why `\d` is written `\\d` in code.

TRY IT

Use `regex_match` with `^[0-9]+$` to check whether three different inputs contain only digits.

CHAPTER

16 Networking & HTTP

Talk to web services and raw TCP servers.

> **SAFETY**
> Use networking features only with systems you own or are explicitly authorized to test. Connecting to other people's services without permission can be illegal.

HTTP

`http_get(url, headers={})`

Sends an HTTP GET request and returns a dictionary with `status`, `body` and `headers`.

EXAMPLE.EHX

```
set response = http_get("https://api.github.com/zen")
show("API Response Code:", response["status"])
show("API Body Content:", response["body"])
```

`http_post(url, body, headers={}, content_type="application/json")`

Sends an HTTP POST request with a body. The content type defaults to JSON.

EXAMPLE.EHX

```
set body = to_json({"name": "ehx", "version": 1})
set reply = http_post("https://api.example.com/items", body)
show("Status:", reply["status"])
```

TCP sockets

`tcp_connect(host, port, timeout=5.0)` opens a raw TCP connection and returns a socket object with three methods: `.send(data)`, `.recv(buffer=4096)` and `.close()`.

EXAMPLE.EHX

```
set sock = tcp_connect("scanme.nmap.org", 80, 5.0)
sock.send("GET / HTTP/1.1\r\nHost: scanme.nmap.org\r\n\r\n")
set response_data = sock.recv(1024)
sock.close()
show("TCP data exchange completed.")
```

TIP

Wrap socket code in `try` / `catch`, set a short timeout, and always call `.close()` when finished.

TRY IT

Call `http_get` on a URL you control and print only the `status` field.

CHAPTER 17

Concurrency & Flow Helpers

Run work in the background and schedule it.

```
spawn(func, ...args)
```

Runs a function in a background thread and returns a task object.

EXAMPLE.EHX

```
fn background_worker(id):
    pause(0.5)
    return f"Worker {id} finished"

set task = spawn(background_worker, 42)
show("Waiting for background task...")
show(task.wait())
```

Task methods

METHOD	DESCRIPTION
`.wait()`	Blocks until the task finishes, then returns its value
`.is_done()`	true if the task has already finished

EXAMPLE.EHX

```
fn async_scan():
    pause(1.0)
    return "Scan results verified"

set scan_task = spawn(async_scan)

while not scan_task.is_done():
    show("Working in background...")
    pause(0.2)

show("Final Task Output:", scan_task.wait())
```

Timers and fatal errors

FUNCTION	DESCRIPTION	EXAMPLE
`set_timeout(fn, delay_ms)`	Calls a function once after a delay in milliseconds	`set_timeout(on_done, 1000)`
`set_interval(fn, ms)`	Calls a function repeatedly, every N milliseconds	`set_interval(pulse, 500)`
`panic(message)`	Halts execution immediately. Use only for unrecoverable errors	`panic("Fatal crash")`

TRY IT

Spawn two workers that each pause for a different time, then `wait()` for both and show their results.

KEY TAKEAWAYS

- `spawn` runs work in the background; `wait()` collects its result.
- `is_done()` lets you poll without blocking.
- `set_timeout` fires once, `set_interval` repeats, `panic` stops everything.

PART V

Practice

Good habits, safety notes, and four small projects built from what you have learned.

IN THIS PART

18 Practical Patterns & Safety

19 Mini Projects

CHAPTER

18 Practical Patterns & Safety

Reliable habits for building real tools.

Build small, verify often

Start with one built-in function, test it with a small value, inspect the output, and then combine it with other features. Short examples make errors easy to locate.

A simple file-log pattern

EXAMPLE.EHX

```
save("audit.log", "[INIT] Script started\n")
add_to("audit.log", "[EVENT] Task completed\n")
set log_contents = read("audit.log")
show(log_contents)
```

Input and type conversion

EXAMPLE.EHX

```
set port_text = ask("Enter a port number: ")

when is_nan(port_text):
    show_err("That is not a number.")
otherwise:
    set port_number = int(port_text)
    show("Selected port:", port_number)
```

Failing gracefully

EXAMPLE.EHX

```
try:
    set page = http_get("https://api.github.com/zen")
    show(page["body"])
catch err:
    show_err("Request failed:", err)
```

Responsibility checklist

- Connect only to systems you own or have clear permission to use.
- Review commands before running them with `exec`.
- Keep real passwords, keys and tokens out of source code and examples.
- Treat encoding and general-purpose hashes as tools for data handling, not password storage.
- Validate input with `is_nan`, `kind` and `has_file` before acting on it.

KEY TAKEAWAYS

- Build in small steps and verify each one.
- Validate input, then convert it.
- Use `try` / `catch` around anything that touches the network or disk.

CHAPTER 19

Mini Projects

Four small programs that combine what you have learned.

Project 1: Log analyzer

Read a log file, keep only the lines that contain `ERROR`, and report how many were found.

LOG_ANALYZER.EHX

```
fn is_error(line):
    return line.contains("ERROR")

when has_file("audit.log"):
    set entries = lines("audit.log")
    set errors = entries.filter(is_error)
    show(f"Errors found: {len(errors)}")
    errors.for_each(show)
otherwise:
    show_err("audit.log not found")
```

Uses: `has_file`, `lines`, `filter`, `for_each`, `len`, f-strings.

Project 2: Local port checker

Check a handful of ports on your own machine, one background task per port.

PORT_CHECKER.EHX

```
set host = "127.0.0.1"
set ports = [22, 80, 443, 8080]

fn check(port):
    try:
        set sock = tcp_connect(host, port, 1.0)
        sock.close()
        return f"Port {port}: open"
    catch err:
        return f"Port {port}: closed"

set tasks = []
for p in ports:
    tasks.add(spawn(check, p))

for t in tasks:
    show(t.wait())
```

SAFETY

Run this only against your own computer or systems you have permission to test.

Uses: `tcp_connect`, `try` / `catch`, `spawn`, `wait`, lists, f-strings.

Project 3: File integrity checker

Fingerprint a file with SHA-256, change it, and detect that something is different.

INTEGRITY.EHX

```
set path = "report.txt"
save(path, "Quarterly report v1")

set original = sha256(read(path))
show("Baseline:", original)

add_to(path, " (edited)")
set current = sha256(read(path))

when current == original:
    show("File unchanged")
otherwise:
    show_err("WARNING: file has changed!")
```

Uses: `save`, `read`, `add_to`, `sha256`, comparison, `when` / `otherwise`.

Project 4: Config-driven tool

Save settings as JSON, load them back, and use them to drive the program.

CONFIG_TOOL.EHX

```
set settings = {"host": "127.0.0.1", "port": 8080, "debug": true}
save("settings.json", to_json(settings, 2))

set loaded = from_json(read("settings.json"))

when loaded.has("port"):
    show(f"Starting on {loaded['host']}:{loaded['port']}")
otherwise:
    show_err("No port configured")
```

Uses: dictionaries, `to_json`, `from_json`, `save`, `read`, `has`.

TRY IT

Extend Project 1 to count lines containing `WARNING` as well, and save a one-line summary with `save`.

APPENDIX

Complete Function Reference

A

Every built-in function and method in EHXLang v1.0.0.

Console, I/O & Formatting

FUNCTION	DESCRIPTION	EXAMPLE
`show(...args)`	Print values to stdout, space-separated	`show("Target:", host, port)`
`show_err(...args)`	Print error messages to stderr	`show_err("Failed to connect")`
`ask(prompt="")`	Read keyboard input from the user	`set pass = ask("Password: ")`
`args()`	List of command-line arguments	`set flags = args()`
`format(template, ...args)`	Replace {} with arguments	`format("IP: {}, Port: {}", ip, 80)`

Type Checking & Conversion

FUNCTION	DESCRIPTION	EXAMPLE
`kind(val)`	Type name as a string	`when kind(x) == "string":`
`len(val)`	Size of a string, list or dict	`set count = len(ports)`
`str(val)`	Convert to string	`str(8080) # "8080"`
`int(val)`	Parse to integer	`int("8080") # 8080`
`float(val)`	Parse to decimal	`float("3.14") # 3.14`
`bool(val)`	Convert to true / false	`bool(1) # true`
`is_nan(val)`	true if not a valid number	`is_nan("abc") # true`

Math Primitives & Extremes

FUNCTION	DESCRIPTION	EXAMPLE
`lowest(...args | list)`	Lowest value	`lowest(10, 4, 99) # 4`
`highest(...args | list)`	Highest value	`highest([10, 4, 99]) # 99`
`low(...args | list)`	Alias for lowest	`low(50, 10, 80) # 10`
`high(...args | list)`	Alias for highest	`high(50, 10, 80) # 80`
`math_abs(x)`	Absolute value	`math_abs(-42) # 42`
`math_sqrt(x)`	Square root	`math_sqrt(64) # 8.0`
`math_pow(base, exp)`	Power	`math_pow(2, 8) # 256.0`
`math_floor(x)`	Round down	`math_floor(4.9) # 4`
`math_ceil(x)`	Round up	`math_ceil(4.1) # 5`
`math_round(x)`	Round to nearest	`math_round(4.6) # 5`
`math_random(low=0, high=1)`	Random float in [low, high]	`math_random(0, 10)`
`math_randint(low, high)`	Random integer, inclusive	`math_randint(1000, 9999)`

File System & Automation

FUNCTION	DESCRIPTION	EXAMPLE
`save(path, data)`	Write or overwrite a file	`save("output.txt", "Ready")`
`read(path)`	Read a whole file as text	`set log = read("app.log")`
`add_to(path, data)`	Append data to a file	`add_to("events.log", "OK\n")`
`has_file(path)`	true if the file exists	`when has_file("conf.json"):`
`lines(path)`	File as a list of lines	`lines("words.txt")`
`pause(seconds)`	Sleep for N seconds	`pause(1.5)`
`exec(command)`	Run an OS command; {code, stdout, stderr}	`set r = exec("whoami")`
`env(key, default="")`	Read an environment variable	`env("USERNAME", "guest")`
`time_now()`	Formatted current date and time	`show(time_now())`
`timestamp()`	Epoch time as a float	`set start = timestamp()`

Cryptography, Archiving & Regex

FUNCTION	DESCRIPTION	EXAMPLE
`md5(text)`	MD5 hex digest	`md5("admin")`
`sha1(text)`	SHA-1 hex digest	`sha1("admin")`
`sha256(text)`	SHA-256 hex digest	`sha256("admin")`
`sha512(text)`	SHA-512 hex digest	`sha512("admin")`
`base64_encode(text)`	Encode to Base64	`base64_encode("secret")`
`base64_decode(encoded)`	Decode Base64	`base64_decode(enc)`
`to_json(data, indent=none)`	Serialize to JSON text	`to_json({"ok": true}, 2)`
`from_json(json_str)`	Parse JSON text	`from_json(raw)`
`zip_pack(source, zip_path)`	Compress to a .zip	`zip_pack("logs/", "bak.zip")`
`zip_unpack(zip_path, dir)`	Extract a .zip	`zip_unpack("bak.zip", "out/")`
`regex_match(pattern, text)`	true if the full text matches	`regex_match("^[0-9]+$", s)`
`regex_find(pattern, text)`	List of all matches	`regex_find("\\d+", s)`
`regex_replace(pat, rep, txt)`	Replace all matches	`regex_replace("[0-9]", "#", s)`

Networking & HTTP

FUNCTION	DESCRIPTION	EXAMPLE
`tcp_connect(host, port, timeout=5.0)`	Open a raw TCP socket	`tcp_connect("127.0.0.1", 80)`
`http_get(url, headers={})`	GET; returns {status, body, headers}	`http_get("https://site.com")`
`http_post(url, body, headers={}, content_type="application/json")`	POST with a body	`http_post(url, body)`

Concurrency & Flow

FUNCTION	DESCRIPTION	EXAMPLE
`spawn(func, ...args)`	Run in a background thread; returns a task	`spawn(worker_fn, 80)`
`set_timeout(fn, delay_ms)`	Call once after a delay	`set_timeout(on_done, 1000)`
`set_interval(fn, ms)`	Call every N milliseconds	`set_interval(pulse, 500)`
`range(start, [stop], [step])`	List of numbers for loops	`for i in range(1, 10):`
`panic(message)`	Halt immediately with an error	`panic("Fatal crash")`

Data type methods

List methods

METHOD	DESCRIPTION	EXAMPLE
`.add(item)`	Append to the end	`ports.add(443)`
`.rm()`	Remove and return last	`set p = ports.rm()`
`.order()`	Sort in place	`ports.order()`
`.fuse(sep="")`	Join into a string	`ports.fuse(" | ")`
`.len()`	Element count	`ports.len()`
`.clear()`	Remove all elements	`ports.clear()`
`.contains(item)`	Membership test	`ports.contains(80)`
`.reverse()`	Reverse in place	`ports.reverse()`
`.slice(start, [end], [step])`	Sub-list	`ports.slice(0, 3)`
`.map(fn)`	Transform elements	`ports.map(double_fn)`
`.filter(fn)`	Keep where fn is true	`ports.filter(is_open)`
`.for_each(fn)`	Run fn on each	`ports.for_each(show)`

String methods

METHOD	DESCRIPTION	EXAMPLE
`.find(sub)`	Index or -1	`"ehx lang".find("lang") # 4`
`.count(sub)`	Occurrences	`"banana".count("a") # 3`
`.len()`	Character count	`"ehx".len() # 3`
`.upper()`	Uppercase	`"ehx".upper()`
`.lower()`	Lowercase	`"EHX".lower()`
`.trim()`	Strip whitespace	`" text ".trim()`
`.split(sep=" ")`	Split into a list	`"10.0.0.1".split(".")`
`.contains(sub)`	Substring test	`s.contains("admin")`
`.starts_with(sub)`	Prefix test	`url.starts_with("https")`
`.ends_with(sub)`	Suffix test	`f.ends_with(".ehx")`
`.replace(old, new)`	Replace all	`s.replace(".", "_")`
`.slice(start, [end], [step])`	Substring	`"hello".slice(0, 4)`

Dictionary methods

METHOD	DESCRIPTION	EXAMPLE
`.keys()`	List of keys	`config.keys()`
`.values()`	List of values	`config.values()`
`.len()`	Pair count	`config.len()`
`.has(key)`	Key exists?	`config.has("port")`
`.clear()`	Remove everything	`config.clear()`

TCP socket methods

METHOD	DESCRIPTION	EXAMPLE
`.send(data)`	Send raw text	`sock.send("GET / HTTP/1.1\r\n\r\n")`
`.recv(buffer=4096)`	Read response data	`sock.recv(1024)`
`.close()`	Close the socket	`sock.close()`

Background task methods

METHOD	DESCRIPTION	EXAMPLE
`.wait()`	Block until done; returns value	`set res = task.wait()`
`.is_done()`	true if finished	`when task.is_done():`

APPENDIX

Quick Lookup Index

B

Find the right tool for the job.

TASK	EHXLANG REFERENCE
`Print output`	show(...args)
`Print errors`	show_err(...args)
`Read user input`	ask(prompt="")
`Read CLI arguments`	args()
`Format a message`	format(template, ...args) or f"..."
`Inspect type / length`	kind(val), len(val)
`Convert values`	str, int, float, bool
`Validate numbers`	is_nan(val)
`Minimum / maximum`	lowest, highest, low, high
`Math & random`	math_abs, math_sqrt, math_pow, math_round, math_floor, math_ceil, math_random, math_randint
`File access`	save, read, add_to, has_file, lines
`System state / time`	env, time_now, timestamp, pause, exec
`Hashing / encoding`	md5, sha1, sha256, sha512, base64_encode, base64_decode
`JSON / archives`	to_json, from_json, zip_pack, zip_unpack
`Patterns`	regex_match, regex_find, regex_replace
`HTTP / TCP`	http_get, http_post, tcp_connect
`Background tasks`	spawn, wait(), is_done()
`Timers / fatal errors`	set_timeout, set_interval, panic
`Lists`	add, rm, order, fuse, contains, reverse, slice, map, filter, for_each
`Strings`	find, count, upper, lower, trim, split, contains, starts_with, ends_with, replace, slice
`Dictionaries`	keys, values, has, len, clear

APPENDIX

Glossary

C

Plain-English definitions of the terms used in this book.

TERM	MEANING
Argument	A value passed into a function when you call it.
AST	Abstract Syntax Tree. The structured form the interpreter builds from your source code before running it.
Base64	A reversible way to represent data as text. It is encoding, not encryption.
Dictionary	A collection of key and value pairs, written with curly braces.
Environment variable	A named setting provided by the operating system, read with `env`.
f-string	A string starting with `f` that evaluates `{expressions}` inside it.
Function	A named, reusable block of code created with `fn`.
Hash	A fixed-size fingerprint of data produced by a one-way function such as SHA-256.
Interpreter	The program (`ehx`) that reads and executes EHXLang code.
JSON	A text format for structured data such as dictionaries and lists.
List	An ordered collection of values, written with square brackets. Also called an array.
Module	An `.ehx` file whose functions can be brought into another file with `load`.
REPL	Read-Eval-Print Loop. An interactive shell started with `ehx repl`.
Regular expression	A pattern describing a set of strings, used with `regex_*` functions.
Socket	An open network connection created by `tcp_connect`.
Task	A background thread started with `spawn`; use `wait()` to get its result.
Timeout	The maximum time to wait for an operation before giving up.

Keep learning.
Keep building.
Build responsibly.

EHXLang v1.0.0 · Created & designed by Yonatan Muluken · ETHIO HACKER X

WRITE LESS. BUILD MORE.

Programming in plain English.

EHXLang replaces complex syntax with clear, active instructions: **set**, **when**, **show**, **save**. This book is your complete companion to the language, from your first line of code to concurrent network tools.

Learn
19 chapters across five parts, with runnable examples.

Build
Four small projects: logs, ports, integrity checks, config.

Look up
Every built-in function and method in one reference.

Stay safe
Responsible-use notes for networking and system features.

CREATED & DESIGNED BY

Yonatan Muluken

ETHIO HACKER X

Telegram @EthioHackerX YouTube @EthioHackerX TikTok @ethio_hacker_x
github.com/EthioHackerX

EHXLang v1.0.0 · MIT License · © 2026 Yonatan Muluken